



Escuela Politécnica Nacional

Nombre del Estudiante: Alexis Sotomayor

Fecha: 22/01/2026

Examen 002

Implementación una API RESTful básica en NestJS, conectada a una base de datos SQLite mediante TypeORM, con relación 1 a muchos (ejemplo: un equipo tiene muchos jugadores) basandose en el ejemplo del proyecto de primer bimestre.

Instrucciones para los estudiantes

1. Repositorio

- Usar el repositorio del curso en la organización de GitHub.
- Crear una carpeta llamada Examen-Web-001 dentro de su misma carpeta de estudiante.
- Subir todo el código fuente del examen en esa carpeta.

2. Configuración del Proyecto

- Crear un proyecto NestJS (nest new examen-web-002).
- Instalar dependencias necesarias:
- Configurar conexión a SQLite en app.module.ts:

3. Definir Entidades

- **Team:** id, name, country.
- **Player:** id, name, position, teamId.
- Relación: un Team tiene muchos Players.

4. Endpoints RESTful Implementar controladores con los siguientes métodos:

- **Teams**
 - GET /teams → obtener todos los equipos
 - GET /teams/:id → obtener un equipo por ID
 - POST /teams → crear un equipo
 - PUT /teams/:id → actualizar un equipo
 - DELETE /teams/:id → eliminar un equipo
- **Players**
 - GET /players → obtener todos los jugadores
 - GET /players/:id → obtener un jugador por ID



- POST /players → crear un jugador
- PUT /players/:id → actualizar un jugador
- DELETE /players/:id → eliminar un jugador
- GET /teams/:id/players → obtener jugadores de un equipo específico

5. README.md

- Instrucciones para instalar dependencias (npm install).
- Cómo correr el servidor (npm run start:dev).
- Ejemplos de endpoints (con curl o HTTPie).

Criterios de Evaluación

- Proyecto correctamente subido al repositorio del curso.
- Conexión a SQLite configurada y funcionando.
- Entidades bien definidas con relación 1 a muchos.
- Endpoints RESTful implementados (crear, modificar, buscar uno, eliminar, buscar muchos).
- README claro y completo.

Desarrollo de la practica

Para la implementación de la API RESTful bajo el framework NestJS, se siguió una metodología estructurada en fases, asegurando la escalabilidad y mantenibilidad del código.

- **Configuración del Entorno y Proyecto:** Se inició generando un nuevo proyecto utilizando el CLI de NestJS (nest new examen-web-002). Posteriormente, se instalaron las dependencias necesarias para la persistencia de datos y documentación:

@nestjs/typeorm, typeorm, sqlite3 y @nestjs/swagger.

- **Implementación de la Capa de Datos:** Se configuró el módulo

TypeOrmModule

en la raíz del proyecto para conectar con una base de datos SQLite local. Se definieron dos entidades principales,



Team (Equipo) y

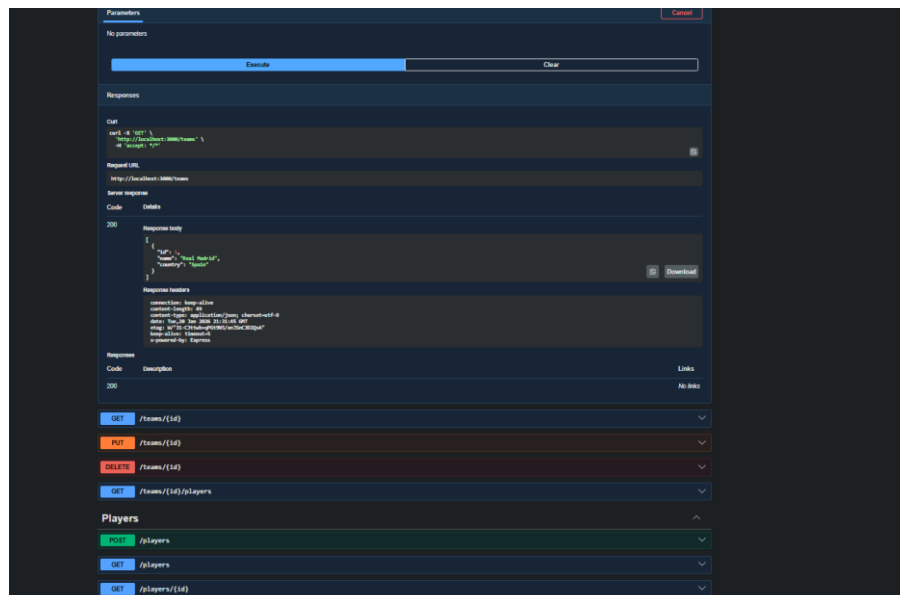
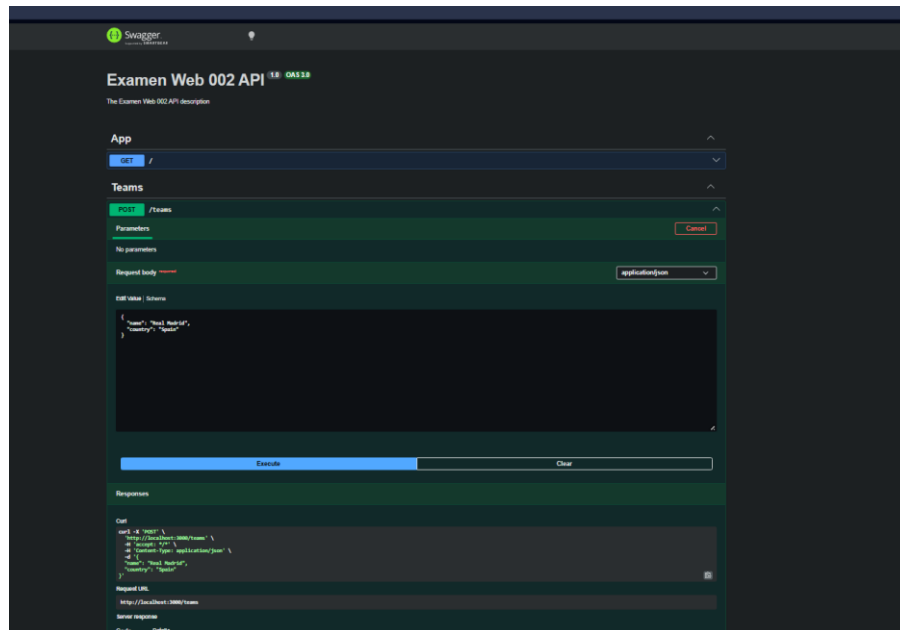
Player (Jugador), estableciendo una relación *uno a muchos* (One-to-Many). Esto se logró mediante el uso de los decoradores @OneToMany en la entidad del equipo y @ManyToOne en la entidad del jugador, garantizando la integridad referencial.

- **Desarrollo de la Lógica de Negocio (CRUD):** Utilizando las capacidades de generación de código de NestJS, se crearon los módulos, controladores y servicios para ambos recursos. Se implementaron los métodos estándar HTTP (GET, POST, PUT, DELETE) para gestionar el ciclo de vida completo de los datos. Se añadieron DTOs (Data Transfer Objects) para tipar y validar la información entrante.
- **Documentación Interactiva:** Finalmente, se integró Swagger (OpenAPI) en el archivo main.ts. Se configuró el DocumentBuilder para generar una interfaz visual accesible vía web, y se documentaron los DTOs con decoradores @ApiProperty para mostrar ejemplos claros de los esquemas de datos requeridos por la API
- **Implementación de la Capa de Datos:** Se configuró el módulo TypeOrmModule en la raíz del proyecto para conectar con una base de datos SQLite local. Se definieron dos entidades principales, Team (Equipo) y Player (Jugador), estableciendo una relación uno a muchos (One-to-Many). Esto se logró mediante el uso de los decoradores @OneToMany en la entidad del equipo y @ManyToOne en la entidad del jugador, garantizando la integridad referencial.
- **Desarrollo de la Lógica de Negocio (CRUD):** Utilizando las capacidades de generación de código de NestJS, se crearon los módulos, controladores y servicios para ambos recursos. Se implementaron los métodos estándar HTTP (GET, POST, PUT, DELETE) para gestionar el ciclo de vida completo de los datos. Se añadieron DTOs (Data Transfer Objects) para tipar y validar la información entrante.
- **Documentación Interactiva:** Finalmente, se integró Swagger (OpenAPI) en el archivo main.ts. Se configuró el DocumentBuilder para generar una interfaz visual accesible vía web, y se documentaron los DTOs con



decoradores @ApiProperty para mostrar ejemplos claros de los esquemas de datos requeridos por la API.

Implementación de la documentación de la API





Análisis de Resultados

Una vez completada la implementación y desplegado el servidor en el entorno local (puerto 3000), se obtuvieron los siguientes resultados funcionales y técnicos:

- **Funcionalidad de la API:** Los endpoints responden correctamente a las solicitudes. Es posible crear un equipo y posteriormente asignar jugadores a dicho equipo utilizando su ID como clave foránea (teamId). Las operaciones de lectura recuperan la información jerárquica esperada.
- **Interfaz de Documentación:** La integración de Swagger en la ruta /api resultó exitosa. Proporciona una interfaz gráfica que permite probar los endpoints directamente desde el navegador sin necesidad de herramientas externas como Postman, facilitando la validación inmediata de los DTOs y los códigos de respuesta.
- **Persistencia de Datos:** La base de datos SQLite almacena correctamente la información. Al reiniciar el servidor, los datos persisten, y las restricciones de la base de datos (como los tipos de datos de las columnas) se respetan gracias al mapeo objeto-relacional de TypeORM.



Conclusiones

- La arquitectura modular de **NestJS** facilitó enormemente la separación de responsabilidades, permitiendo desarrollar los módulos de Teams y Players de manera independiente pero cohesionada a través de la inyección de dependencias.
- El uso de **TypeORM** abstraigo la complejidad de las sentencias SQL, permitiendo manipular la base de datos utilizando objetos y clases. La definición de relaciones mediante decoradores simplificó la lógica para vincular jugadores con sus respectivos equipos.
- La implementación de **Swagger** no solo sirve como documentación técnica, sino que mejora la experiencia de desarrollo al proporcionar un entorno de pruebas en tiempo real, lo cual es crucial para verificar la correcta estructura de los datos de entrada y salida (DTOs).
- En general, la práctica demostró cómo un stack tecnológico moderno (Node.js + NestJS + ORM) permite construir APIs robustas y escalables con una inversión de tiempo razonable gracias a la automatización y las herramientas de desarrollo incluidas.

NOTA: La implementación de los ejemplos del consumo del proyecto para el Backend para las rutas de **Teams y Players**